

🔥 Awesome — let's start **Week 5 Day 1 (properly, practical + aligned to YOUR system)**

🚀 ✅ **Week 5 Day 1 — BM25 Keyword Search**

🧠 **BLOCK 1 — DSA Concept (Simple understanding)**

✅ **Idea: Trie vs Inverted Index**

You saw Trie:

```
node.children = {}
```

👉 works like:

```
"d" → "da" → "dat" → "data"
```

✅ Good for prefix search

❌ But NOT good for document retrieval

✅ **What BM25 uses (IMPORTANT)**

👉 Instead of Trie:

HashMap:

```
word → [chunk1, chunk2, chunk3]
```

Example:

```
"binary" → [chunk2, chunk7]
```

```
"search" → [chunk2, chunk9]
```

👉 This is called **Inverted Index**

⚡ **Difference vs YOUR current system**

System	Works on
ChromaDB	meaning (semantic)

System	Works on
BM25	exact words (keywords)

ONE-LINE

- ✓ BM25 = "match exact words"
- ✓ Chroma = "match meaning"

BLOCK 2 — AI Concept (BM25)

✓ What BM25 does

For query:

binary search

BM25 gives higher score if:

- ✓ Words appear often
- ✓ Words are rare overall
- ✓ Chunk is not too long

✓ Simple code usage

```
from rank_bm25 import BM25Okapi
```

```
tokenized = [chunk.lower().split() for chunk in chunks]
```

```
bm25 = BM25Okapi(tokenized)
```

```
query_tokens = "binary search".split()
```

```
scores = bm25.get_scores(query_tokens)
```

✅ **Output**

```
scores = [0.2, 5.6, 0.1, 7.2]
```

👉 Bigger score = more relevant

🔥 **BLOCK 3 — BUILD (your project integration)**

✅ **STEP 1 — install**

```
pip install rank-bm25
```

✅ **STEP 2 — Create BM25 class**

Add in your DocumentLoader file:

```
from rank_bm25 import BM25Okapi
```

```
class BM25Search:
```

```
    def init(self):
```

```
        self.bm25 = None
```

```
        self.chunks = []
```

```
    def build(self, chunks):
```

```
        self.chunks = chunks
```

```
        tokenized = [chunk.lower().split() for chunk in chunks]
```

```
        self.bm25 = BM25Okapi(tokenized)
```

```
def search(self, query, k=3):  
    query_tokens = query.lower().split()  
    scores = self.bm25.get_scores(query_tokens)  
  
    top_indices = sorted(range(len(scores)),  
                        key=lambda i: scores[i],  
                        reverse=True)[:k]  
  
    return [self.chunks[i] for i in top_indices]
```

✅ STEP 3 — Attach to your loader

Inside DocumentLoader.__init__:

```
self.bm25 = BM25Search()
```

✅ STEP 4 — Build index after chunking

Inside embed_all_chunks():

```
self.vector_store.add_chunks(self.chunks, embed_fn)
```

✅ ADD THIS

```
self.bm25.build(self.chunks)
```

✅ STEP 5 — Test BM25 separately

Add temporary debug:

```
print("BM25 result:", self.loader.bm25.search("binary search"))
```

✅ Expected result

```
[  
  "Binary search is an algorithm...",  
  "It works by dividing the array..."  
]
```

🔥 ✅ Compare with ChromaDB

🔵 ChromaDB gives:

related ideas (BST, searching, trees)

🟢 BM25 gives:

exact keyword matches

🧠 KEY Insight

👉 Your earlier issue:

vector search returned noise

👉 BM25 fixes:

exact matches only ✅

✅ ✅ VERIFICATION CHECKLIST

Make sure:

- ✅ BM25 index builds
 - ✅ search() returns chunks
 - ✅ results contain exact keywords
 - ✅ faster than vector search
-

REFLECTION (IMPORTANT — answer mentally)

1. BM25 vs vector:

- Which gave better "binary search" answer?

2. Query:

fast lookup table

👉 BM25 fails ❌

👉 vector works ✅

✅ WHY? → semantic understanding

FINAL UNDERSTANDING

✅ BM25 = precision

✅ Vector = understanding

✅ Tomorrow = combine both = POWER 🔥

NEXT

👉 Say:

"Week 5 Day 2"

I'll upgrade your system to **HYBRID SEARCH (big milestone)** 🚀